

NYX Engine v1.0

SPEKIFIKASI API SISTEM MANAJEMEN MULTI-TENANCY & PANDUAN INTEGRASI SDK SDK KLIEN 1-ON-1

Klasifikasi Dokumen: Core Developer & Partner Integration Specification

Prinsip Keamanan: Zero-Knowledge, Post-Quantum Cryptography Hardened (X-Wing KEM & ML-KEM-768)

Status Rilis: Production Ready (Stable Edition)

1. Arsitektur Komunikasi NYX Engine (B2B BaaS)

NYX Engine menyediakan infrastruktur **Backend-as-a-Service (BaaS) Multi-Tenancy** yang aman, memungkinkan aplikasi pihak ketiga (Tenant B2B, seperti web marketplace, customer service, atau sistem joki) untuk mengintegrasikan ruang obrolan terenkripsi penuh tanpa mengekspos data mentah pesan atau kunci enkripsi ke server penengah.

Protokol yang diadopsi khusus untuk integrasi B2B (Iframe maupun SDK) adalah murni **Post-Quantum Double Ratchet (PQ-DR) 1-on-1 bertipe terisolasi**. Berbeda dengan aplikasi B2C NYX organik yang mendukung multi-device lewat *Fanout Sender Key*, sesi B2B sengaja dirancang **tanpa dukungan multi-device** demi mematuhi aspek akuntabilitas dan efisiensi memori (Zero-Overhead). Sesi obrolan diikat murni pada satu instansiasi perangkat lokal yang dinamis (Session-Bound).

2. Langkah demi Langkah Mendapatkan Akses API (S2S Authentication)

Sebelum server pihak ketiga (Tenant) dapat menginisialisasi pembuatan ruangan chat, server tersebut wajib memiliki kunci otentikasi utama yang sah. Berikut adalah alur siklus otentikasi dari hulu ke hilir:

Langkah 2.1: Registrasi Tenant via Admin Dashboard

1. Masuk ke dalam aplikasi NYX Messenger menggunakan akun administrator yang memiliki hak akses penuh.
2. Buka halaman **Admin Dashboard** dan navigasikan ke tab **B2B Engine / Tenants**.
3. Masukkan nama aplikasi Tenant baru Anda (contoh: **JokiApp Marketplace**) dan tentukan domain yang diizinkan untuk memuat Iframe Anda di kolom **Allowed Domains** (pisahkan dengan koma jika lebih dari satu, contoh: **https://jokiapp.com, http://localhost:5173**).
4. Klik tombol **Generate API Key**. Sistem backend akan mengeksekusi fungsi kriptografi acak yang menghasilkan string heksadesimal 32-byte yang unik secara deterministik.
5. Salin API Key tersebut dan simpan dengan aman di dalam file konfigurasi lingkungan (**.env**) pada server backend Anda sebagai variabel **NYX_ENGINE_KEY**.

Langkah 2.2: Melakukan Server-to-Server (S2S) Room Provisioning

Setiap kali terjadi transaksi di aplikasi Anda yang membutuhkan ruangan chat aman (misal: pembeli membeli slot joki dan harus berinteraksi dengan joki), server backend Anda wajib menembak endpoint NYX Engine untuk membuat ruangan terisolasi.

Endpoint HTTP: **POST** `/api/engine/rooms`

Headers Wajib: `x-nyx-engine-key: [API_KEY_TENANT_ANDA]`

Contoh Payload Request (JSON):

```
{
  "userA": {
    "externalId": "buyer_user_9912",
    "displayName": "Budi Gaming"
  },
  "userB": {
    "externalId": "joki_pro_007",
    "displayName": "Alex PushRank"
  },
  "metadata": {
    "orderId": "ORD-ML-2026-X91",
    "serviceContext": "Mobile Legends Mythic Push"
  }
}
```

Contoh Response Sukses (200 OK):

```
{
  "userAUrl": "https://app.nyx-app.my.id/embed/chat/cmpeaf2jk00crzs?token=eyJhbGciOiJIUzI...\"",
  "userBUrl": "https://app.nyx-app.my.id/embed/chat/cmpeaf2jk00crzs?token=eyJhbGciOiJIUzI...\""
}
```

Catatan Sistem Kripto Di Balik Layar:

Backend NYX secara otomatis mengamankan identitas dengan memproses kombinasi `tenantId` dan `externalId` menggunakan algoritma SHA-256 untuk menghasilkan `usernameHash` yang deterministik dan unik per tenant. Jika user belum terdaftar, sistem membuatnya entitas user baru secara senyap (upsert) dengan password acak aman 32-byte berbasis Argon2id.

3. Metode Integrasi Jalur A: High-Security Iframe (Embed Mode)

Ini adalah metode integrasi tercepat yang tidak membutuhkan koding UI di sisi klien Anda. Anda cukup menempelkan tautan (URL) sakti yang dikembalikan oleh respons S2S API ke dalam elemen HTML `<iframe>` di dasbor masing-masing pengguna.

Langkah 3.1: Pastikan Konfigurasi Gerbang Nginx Mengizinkan Cross-Frame Loading

Secara default, proteksi CSP NYX melarang aplikasi dimuat di dalam bingkai lain (`frame-ancestors 'none'`). Anda harus memastikan file `web/nginx.conf` di server VPS NYX Anda telah diperbarui untuk mengizinkan pemuatan bingkai:

```
# Ganti frame-ancestors 'none'; menjadi simbol asteris (*) untuk kebutuhan localhost,  
# atau batasi murni ke domain milik klien Anda demi keamanan optimal.  
add_header Content-Security-Policy "default-src 'self'; frame-ancestors https://  
jokiapp.com http://localhost:5173;"
```

Langkah 3.2: Pemasangan Tag HTML Iframe pada Aplikasi Frontend Anda

Di halaman dasbor Pembeli (User A), sematkan kode berikut menggunakan nilai `userAUr1` :

```
<iframe  
  src="https://app.nyx-app.my.id/embed/chat/cmpeaf2jk00crzsj?token=eyJhbGciOiJIUzI..."  
  style="width: 100%; height: 600px; border: none; background: #0b0f19;"  
  allow="clipboard-write; storage-access;">  
</iframe>
```

Halaman khusus `/embed/chat/:id` dirancang murni sebagai *Ghost Mode UI*. Komponen ini tidak akan merender sidebar navigasi global, tidak memuat bilah menu settings, dan 100% ukurannya menempati viewport Iframe secara bersih. Token di dalam query parameter ditangkap secara asinkron, disuntikkan ke dalam modul `tokenStorage` , melakukan otentikasi otomatis, membuka gerbang WebSockets, dan menginisialisasi sesi chat terenkripsi secara instan.

4. Metode Integrasi Jalur B: Core Klien SDK Reference

Jika Anda menginginkan kontrol UI penuh dan ingin membangun komponen kotak chat sendiri di dalam aplikasi React/Vue/NextJS murni milik Anda, gunakan paket SDK klien resmi `@nyx-engine/sdk` yang telah diekstraksi.

Langkah 4.1: Instalasi Paket SDK

Letakkan file tarball hasil build SDK (`.tgz`) di dalam direktori proyek Anda, lalu jalankan instalasi menggunakan manajer paket Anda:

```
pnpm install ./nyx-engine-sdk-0.1.0.tgz  
# atau jika menggunakan npm / yarn  
npm install ./nyx-engine-sdk-0.1.0.tgz
```

Langkah 4.2: Inisialisasi dan Autentikasi Pengguna Klien

Impor kelas utama `NyxClient` , lakukan instansiasi dengan menyertakan konfigurasi API Key Tenant, lalu panggil instruksi asinkron koneksi menggunakan JWT Token yang Anda dapatkan dari rute S2S `/rooms` sebelumnya.

```
import { NyxClient } from '@nyx-engine/sdk';

const nyx = new NyxClient({
  apiKey: 'YOUR_TENANT_API_KEY',
  tenantId: 'jokiapp_tenant_prod_1',
  environment: 'production',
  baseUrl: 'https://api.nyx-app.my.id'
});

// 1. Bootstrapping mesin Libsodium WASM lokal dan adapter storage
await nyx.initialize();

// 2. Lakukan koneksi Socket Handshake dan Auto-Device Registration
// Fungsi ini kebal terhadap kondisi race-condition socket connect event.
await nyx.connectUser(tokenFromBackend);

console.log("NYX Engine siap digunakan secara aman!");
```

Langkah 4.3: Mendengarkan Pesan Masuk yang Terdekripsi Otomatis

Anda tidak perlu memikirkan algoritma ratcheting yang rumit. Cukup daftarkan pendengar event (event listener) pada instansiasi objek klien Anda. SDK secara otomatis menangkap cipher pesan dari WebSockets, memutar roda kunci Double Ratchet, dan mengembalikan teks polos (plaintext).

```
nyx.on('message.decrypted', (payload) => {
  console.log(`Pesan masuk terdekripsi dari ruangan [${payload.conversationId}]:`);
  console.log(`Plaintext: ${payload.plaintext}`);

  // Perbarui state UI aplikasi Anda dengan data plaintext ini
  appendMessageToUi(payload.conversationId, payload.plaintext);
});
```

Langkah 4.4: Mengirimkan Pesan Teks

```
try {
  // Cukup sebutkan ID Ruang Percakapan dan string pesan teks polos Anda.
  // SDK otomatis merajut kunci simetris, melakukan enkripsi PQC, lalu menembakkannya.
  await nyx.sendMessage('cmpeaf2jk00crzsj', 'Halo bang, orderan joki saya tolong diproses ya.');
```

```
} catch (error) {
  console.error("Gagal mengirim pesan teks terenkripsi:", error);
}
```

5. Siklus Hidup Pesan & Kriptografi Lampiran (Zero-Knowledge Media)

B2B Tenant wajib mendukung fitur pengiriman bukti berupa gambar (misal screenshot transaksi atau hasil kenaikan rank game). Untuk menjaga prinsip kerahasiaan total (Zero-Knowledge), file media tidak boleh diunggah langsung

dalam bentuk mentah ke peladen awan. Dokumen ini menjelaskan bagan alur komputasi enkripsi file di sisi klien sebelum proses unggah terjadi:

[Image of E2EE Sender Key auto-recovery mechanism showing offline queued request and distribution reply]

Langkah 5.1: Alur Enkripsi Lampiran (Media Outgoing)

Saat fungsi `nyx.sendMedia(conversationId, fileBlob, mimeType, caption)` dipanggil oleh aplikasi, SDK mengeksekusi operasi berikut secara internal tanpa membebani thread antarmuka utama (Main UI Thread):

1. SDK memanggil fungsi utilitas lokal `encryptBlob(fileBlob)`.
2. Instansiasi Libsodium menghasilkan kunci simetris acak 32-byte (XChaCha20-Poly1305) dan Nonce 24-byte baru khusus untuk file tersebut.
3. Blob file mentah dikonversi menjadi ArrayBuffer, dienkripsi penuh di memori lokal, dan dibungkus kembali menjadi objek Blob terenkripsi biner dengan tipe content `application/octet-stream`.
4. SDK menembak REST API backend untuk meminta alamat unggah sementara: `POST /api/uploads/presigned`.
5. SDK melakukan native HTTP `PUT` request membawa Blob terenkripsi langsung menuju penyimpanan cloud (R2/S3 Bucket) menggunakan presigned URL yang diberikan. Langkah ini bersih tanpa menyertakan header otorisasi NYX untuk menghindari isu pemblokiran CORS.
6. Setelah berhasil diunggah, SDK merangkai objek metadata teks berformat JSON yang berisi alamat URL file, kunci simetris rahasia, serta nilai nonce-nya.
7. Objek metadata JSON teks tersebut dimasukkan ke dalam mesin utama `DoubleRatchetEngine` untuk dienkripsi ulang menggunakan roda kunci percakapan saat itu, lalu dilempar lewat Socket.IO.

Contoh Struktur Payload Teks Media yang Terlindungi di Dalam Double Ratchet:

```
{
  "type": "media",
  "caption": "Bukti akun joki sudah mencapai Rank Mythic, silakan dicek.",
  "media": {
    "url": "https://storage.nyx-app.my.id/attachments/file-22a1b9c7.enc",
    "symmetricKey": "d3FhX3NlY3JldF9reXNfcTR3X2VjMmVlX3BxY19reQ",
    "nonce": "eF93aW5nX2tlbV9ub25jZV85MThh",
    "mimeType": "image/png"
  }
}
```

Langkah 5.2: Alur Dekripsi Lampiran (Media Incoming)

Ketika Anda menerima event obrolan yang membawa payload biner bertipe media, peladen NYX hanya mengetahui barisan string acak. Untuk merendernya ke layar pengguna secara polos, Anda wajib memanggil helper dekripsi lampiran yang diekspos oleh SDK publik:

```

import { decryptBlob } from '@nyx-engine/sdk';

// Asumsi 'messagePayload' didapat dari event listener setelah terdekripsi dari DR
if (messagePayload.type === 'media') {
  const { url, symmetricKey, nonce, mimeType } = messagePayload.media;

  // 1. Ambil file biner terenkripsi (ciphertext) dari cloud storage
  const res = await fetch(url);
  const encryptedBlob = await res.blob();

  // 2. Jalankan dekripsi simetris lokal menggunakan kunci rahasia yang terikat di pesan
  const decryptedBlob = await decryptBlob(encryptedBlob, symmetricKey, nonce);

  // 3. Konversi blob menjadi objek URL lokal agar bisa dirender oleh tag HTML <img>
  const localImageSrc = URL.createObjectURL(new Blob([decryptedBlob], { type:
mimeType }));

  renderImageToDashboard(localImageSrc, messagePayload.caption);
}

```

6. Arsitektur Ketangguhan Jaringan & Protokol Antrean Luring

Kondisi jaringan dunia nyata sangatlah fluktuatif (anomali *Network Drops* / RTO). NYX Engine dilengkapi dengan komponen ketangguhan tingkat militer untuk memastikan tidak ada pesan atau putaran roda kunci percakapan yang rusak atau hilang saat pengguna bertransisi dari luring ke daring.

Langkah 6.1: Mekanisme Batas Antrean Offline Outbox

Ketika metode `sendMessage()` dieksekusi, SDK akan memeriksa properti status koneksi fisik melalui `this.socket.connected`. Jika bernilai `false`, pesan terenkripsi tidak akan dibuang atau memunculkan error crash di UI aplikasi Anda.

Pesan otomatis diumpankan ke dalam objek komponen `NyxQueueManager` dan disimpan persisten ke tabel IndexedDB lokal bernama `offline_outbox` dengan skema FIFO (First-In-First-Out) teratur menggunakan pengidentifikasi UUID yang unik. Begitu koneksi internet pulih, internal SDK secara reaktif menangkap event `connect`, mengaktifkan status pengurutan terproteksi mutex (`isFlushing = true`), lalu menembakkan tumpukan pesan luring berurutan ke server tanpa risiko duplikasi atau tumpang tindih kunci.

Langkah 6.2: Protokol Pemulihan Kunci Persisten (Persistent Offline Key Requests)

Penyakit terbesar pada skenario luring-ke-daring (Offline-to-Online Sync) di aplikasi E2EE berbasis kunci grup (Sender Key) adalah ketika user A mengirim pesan beruntun saat user B sedang offline, lalu terjadi rotasi kunci atau pembersihan cache lokal yang memicu error fatal: `pq_box_seal_open ciphertext cannot be decrypted using that key`.

NYX Engine menyelesaikan masalah ini secara elegan melalui arsitektur **Persistent Offline Key Request Protocol**:

- **Deteksi Kegagalan Otomatis:** Jika modul pemroses pesan luring di sisi penerima (Bob) mendeteksi kegagalan unseal akibat ketidakcocokan kunci privat perangkat, sistem tidak akan menghentikan pemrosesan antrean. Sesi menangkap exception secara halus, mengonversi status teks pesan menjadi placeholder aman berupa string token `<Decryption Failed>`, dan menyisipkan penanda `waiting_for_key` di memori lokal database.
- **Pengiriman Pesan Sistem Persisten:** Alih-alih memancarkan event soket sementara yang akan lenyap jika pengirim sedang offline, sistem Bob secara otomatis merajut pesan sistem rahasia bertipe `SYSTEM_KEY_REQUEST` terenkripsi penuh via jalur 1-on-1 normal menuju Alice. Karena berwujud "pesan", server NYX akan menyimpannya secara persisten di database pusat.
- **Redistribusi Kunci secara Senyap:** Saat pengirim (Alice) kembali online, sistemnya mengunduh pesan sistem tersebut secara latar belakang. Logika pengevaluasi kontrol (`evaluateControlMessage`) mendeteksi tipe request kunci, mencegah kemunculannya di antarmuka UI Alice (100% Invisible), mengekstrak kunci grup baru yang sah melalui `emitGroupKeyDistribution`, lalu mengirimkannya kembali ke Bob.
- **Dekripsi Ulang UI yang Reaktif:** Begitu kunci baru yang dinanti tiba di komputer Bob, fungsi `reDecryptPendingMessages` langsung memindai barisan placeholder yang tertunda, memutar kembali perhitungan algoritma dekripsi, dan secara instan memperbarui teks gelembung chat asli di komponen `MessageBubble.tsx` secara reaktif tanpa perlu memicu proses penyegaran (refresh) halaman aplikasi.

7. Panduan Lengkap Troubleshooting & Penanganan Masalah Umum

Tabel di bawah ini memuat rangkuman dokumentasi ranjau error produksi yang sering ditemui oleh developer integrasi beserta solusi mitigasi teknisnya:

Pernyataan Masalah / Log Error	Analisis Akar Masalah (Root Cause)	Tindakan Korektif & Solusi Kode
Error: ciphertext cannot be decrypted using that key FATAL CRYPTO	Terjadi kegagalan unseal kunci simetris pada utas Web Worker. Disebabkan oleh ketidakcocokan antara varian skema pengodean Base64 yang dikirim perangkat lawan (misal menggunakan standard padding) dengan skema pembacaan Libsodium lokal (URLSafe no padding).	Bungkus penanganan unseal biner Anda di dalam fungsi <code>handleGroupKeyDistribution</code> menggunakan blok <code>try-catch</code> berlapis secara sekuensial yang mencoba melakukan parsing semua ragam varian format Base64 (<code>URLSAFE</code> , <code>ORIGINAL</code> , dan <code>URLSAFE_NO_PADDING</code>) sebelum melemparkan status kegagalan.
Layar Iframe Blank Putih / Refused to display '...' in a frame because it violates CSP	Kebijakan keamanan browser memblokir pemuatan halaman NYX di dalam domain eksternal karena header HTTP gerbang Nginx masih menegakkan aturan restriksi ketat <code>frame-ancestors 'none'</code> .	Buka file konfigurasi <code>web/nginx.conf</code> di peladen VPS Anda. Cari variabel aturan Content Security Policy (CSP), ganti nilai parameter <code>frame-ancestors 'none'</code> ; menjadi daftar domain resmi milik server Tenant Anda (contoh: <code>frame-ancestors https://jokiapp.com;</code>). Lakukan reload layanan via terminal: <code>systemctl reload nginx</code> .
Promise Hangs / Aplikasi Klien Freeze saat Memanggil Fungsi connectUser()	Kondisi balapan data (Race Condition). Kode pembungkus Promise menunggu pemicuan event socket <code>this.socket.once('connect')</code> , namun koneksi fisik socket sebenarnya sudah berstatus tersambung penuh sebelum baris kode listener sempat dieksekusi oleh runtime JavaScript.	Perbarui baris logika di dalam file SDK utama Anda (<code>packages/nyx-sdk/src/client.ts</code>). Lakukan pemeriksaan kondisi awal secara sinkron: Jika nilai properti dari getter <code>this.socket.connected</code> sudah bernilai <code>true</code> , langsung eksekusi fungsi handler internal Anda seketika itu juga tanpa memasang objek listener baru.
CORS Error / Gagal Melakukan PUT Request saat Mengunggah Lampiran Media	Penyedia penyimpanan objek awan (Cloudflare R2 atau AWS S3 Bucket) menolak unggahan karena aturan kebijakan CORS di dasbor bucket Anda belum mengizinkan metode HTTP <code>PUT</code> atau belum memperbolehkan header kustom dilewatkan dari luar asal domain.	Masuk ke dalam ruang kontrol dasbor konsol Cloudflare R2 / AWS S3 Anda, buka bagian tab pengaturan konfigurasi CORS pada bucket yang digunakan, lalu pastikan Anda telah menambahkan aturan JSON yang mengizinkan asal domain (<code>AllowedOrigins: ["*"]</code>) dan mengaktifkan metode operasi biner (<code>AllowedMethods: ["PUT", "GET", "POST"]</code>).

Peringatan Kepatuhan Memori Enkripsi:

Ketika Anda mengembangkan kode kustom tambahan menggunakan SDK ini, pastikan seluruh penanganan kunci simetris dan array mentah biner dibungkus di dalam struktur blok `try...finally`. Di dalam cakupan perintah `finally`, Anda **wajib** mengeksekusi metode `sodium.memzero(keyBytes)` untuk secara aktif menghapus jejak memori pada heap buffer browser demi mencegah potensi eksploitasi kebocoran memori (heap storage memory leaks).